1. Time Complexity using Big-O Notation:

Operation	Time Complexity	Explanation
addFirst(name, cgpa)	O(1)	Involves a few pointer updates to insert at the beginning of the list.
addLast(name, cgpa)	O(1)	Adding at the end is constant time if the tail node is maintained.
insertAt(index, name, cgpa)	O(n)	In the worst case, the list needs to be traversed to find the insertion point.
search(name)	O(n)	It requires a linear scan through the list to find the student by name in the worst case
display()	O(n)	Needs a linear traversal to display all nodes.
getSize()	O(1)	The size is maintained as a variable and can be accessed directly.

deleteFirst()	O(1)	Simply involves updating the head pointer.
deleteLast()	O(n)	It requires traversing the list to find the second-to-last node before deletion.
deleteAt(index)	O(n)	Similar to deleteLast, traversal is required to find the node before the target.

1.2 Key Optimizations:

1. Tail Pointer:

- Speeds up addLast() and deleteLast() by providing direct access to the last node.
- Time complexity improves from O(n) to O(1).

2. Dynamic Size Tracking:

- Maintaining a size counter allows getSize() to run in O(1), avoiding traversal.

3. Doubly Linked List:

- Supports bidirectional traversal, making operations like deleteLast() and insertAt() more efficient.

4. Hash Map for Search:

- Using a hash map with names as keys can make search() O(1). This increases space complexity but significantly speeds up retrieval.

2. Performance Analysis for Large Linked Lists (10,000 Nodes)

Efficient Operations:

- addFirst(), deleteFirst(), and getSize() (if size is tracked) perform efficiently, remaining constant time even for large lists.
- addLast() and deleteLast() become efficient if a tail pointer is used.

Inefficient Operations:

- insertAt(), deleteAt(), and search() are O(n) and become slower as the list grows larger.
- Frequent middle insertions or deletions can become bottlenecks in large datasets.

2.1 Comparison: Singly Linked List vs. ArrayList:

Operation	Singly Linked List	ArrayList
Add First	O(1)	O(n)
Add Last	O(n) (O(1) with tail)	O(1)
Insert at Index	O(n)	O(n)
Delete First	O(1)	O(n)
Delete Last	O(n)	O(1)
Delete at Index	O(n)	O(n)
Search	O(n)	O(n)
Access by Index	O(n)	O(1)
Display/Traversal	O(n)	O(n)
Get Size	O(1) (if tracked)	O(1)

2.2 Edge Cases in Singly Linked List Operations:

1. Empty List:

- Insertions (addFirst() or addLast()) create a new node as the head.
- Deletions (deleteFirst() or deleteLast()) should handle empty states gracefully to avoid errors.

2. Non-Existent Search:

- **Best case:** O(1) if the desired element is at the head.
- **Worst case:** O(n) if the element doesn't exist.

3. Single Node List:

- Insertions add a second node at the head or tail, with complexities depending on the operation.
- Deletions remove the only node, leaving the list empty.

2.3 Real-World Application of Linked List:

1. **Dynamic Memory Allocation:** Linked lists are commonly used to manage free memory blocks in operating systems. When a program requests memory, a linked list keeps track of free and allocated memory blocks, allowing efficient allocation and deallocation without requiring contiguous memory.
   Example: Functions like malloc() in C or new in C++ use linked lists for memory management.
2. **Undo/Redo in Applications:** Every activity in programs such as design software or text editors is saved as a state in a linked list. When a user clicks "undo," the program goes back to the list's first node. Redo operations proceed in the list in a similar manner. Changes are easily stored and accessed because of its dynamic framework.

   Example: the ability to undo actions in Adobe Photoshop or Microsoft Word.
3. **Browser History Navigation:** A doubly linked list is ideal for managing browser history. Each web page visited is a node, with links to the previous and next pages. Users can navigate backward or forward easily without the need for large-scale memory adjustments.
   Example: Google Chrome or Mozilla Firefox history tracking.
4. **Music Playlists:** Linked lists allow music playlists to be dynamic. Tracks can be added, removed, or reordered efficiently without shifting other elements in memory, making it suitable for applications that need flexible data management.
   Example: Spotify or VLC Media Player play**l**ist management

3.Conclusion:

In this report, we have discussed the time complexities of different operations of a singly linked list with the help of Big-O notation. Even though a singly linked list is efficient for some operations like adding or removing elements at the head, its performance is poor for operations such as searching or accessing elements by index since it requires traversal. Key optimizations can include maintaining a tail pointer, dynamically tracking size, or the use of a doubly linked list that dramatically improves performance within certain use cases.

Comparing singly linked lists to ArrayLists, it seems the choice of which one to use depends on the application. Applications that require a lot of insertions or deletions should lean towards using singly linked lists, while applications with many random accesses and dynamic resizes at the end can use ArrayLists. In this way, the knowledge of the trade-offs becomes important in choosing the right data structure for the best performance of a particular task.